

Open Agentic Pipeline — Evolving Plan

This document is a living plan. It evolves with each iteration. Lessons learned, failed experiments, and metric changes are recorded here so future development builds on past findings.

Goal

Replace the fixed-pipeline agent (`agent.py`) with a **true agentic classifier** powered by `claude -p` (headless Claude Code). Instead of a hardcoded multi-stage pipeline with prescribed phases, each prediction is an autonomous Claude agent that:

- Receives a test image + access to reference images and symptom KB
- Freely reasons, reads files, and views images using Claude Code's native tools
- Submits a structured prediction when ready

The hypothesis: a free-form reasoning agent with access to symptoms and reference images can outperform both few-shot baselines and the current rigid pipeline, because it can adaptively allocate attention based on diagnostic difficulty.

Architecture

```
Harness (eval.py)
- Loads dataset (train/test splits)
- Loads KB (default / local / internet xlsx)
- Spawns N parallel `claude -p` subprocesses
- Parses stream-json traces
- Computes metrics (accuracy, turns, cost)
- Writes per-image logs + summary
```

↓ one subprocess per test image

```
Claude -p Agent (one per image)
- Model: claude-sonnet-4-6
- Allowed tools: Read (for images + files)
- System prompt: task description + constraints
- User message: test image + class list + KB
- Budget: max K reference image views
- Output: prediction JSON in final response
```

How the agent works

Each `claude -p` call receives:

1. **Test image** — passed as a file path; the agent reads it via the `Read` tool
2. **Class list** — all disease class names (from folder names)

- 3. **Symptom KB** — full symptom text for all classes, embedded in the prompt
- 4. **Reference image paths** — the agent knows where train images live and can **Read** them
- 5. **Budget** — stated in the prompt: "you may view at most K reference images"
- 6. **Output format** — the agent must end its response with a JSON block: `{"prediction": "...", "confidence": 0.0-1.0, "reasoning": "..."}`

The agent is free to reason in any order. No prescribed phases. It can:

- Read symptoms first, then view images
- View images first, then check symptoms
- Compare multiple candidates side-by-side
- Skip classes it's confident about

Key differences from current agent.py

Aspect	Current (agent.py)	Open Agentic
Agent runtime	Anthropic API tool-use loop in Python	claude -p subprocess (Claude Code)
Reasoning structure	5 hardcoded phases (ORIENT→SURVEY→NARROW→CONFIRM→VERIFY)	Free-form, prompt-guided
Tools	Custom Python tool handlers	Claude Code native tools (Read, etc.)
Image access	Base64 inline in API messages	File paths, agent reads via Read tool
Traces	Custom trace list in Python	stream-json NDJSON from claude CLI
Cost tracking	Token counting from API response	total_cost_usd from result event

Eval Configuration (matches run_eval.sh)

Parameter	Value
--symptom-source	none, default, local, internet
--num-classes	5
--images-per-class	5
--k	4 (max reference image views)
--parallel	12
--seed	42
Dataset	Soybean_Diseases (32 classes)

Metrics

- **Accuracy** — primary metric (correct / total)
- **Per-class accuracy** — to identify weak spots
- **Avg turns** (`num_turns` from stream-json result)
- **Avg cost** (`total_cost_usd` from stream-json result)
- **Avg duration** (`duration_ms` from stream-json result)
- **Refs viewed** — count of reference image `Read` calls in trace
- **Error rate** — subprocess failures / timeouts

KB Sources & Comparison Axes

All KB sources are **filtered to the target crop only** (e.g., soybean). Never send the full multi-crop file.

Source	Description	Origin
<code>none</code>	No KB — agent uses only reference images	—
<code>local</code>	PDF-extracted symptom descriptions	<code>disease_registry/outputs/Soybean_local.xlsx</code>
<code>internet</code>	Web-extracted symptom descriptions	<code>disease_registry/outputs/Soybean_internet.xlsx</code>

The comparison tests whether curated KB (local from PDF, internet from web) outperforms images-only (none). GPT-generated KB (`disease_symptoms_crop_wise.md`) is excluded — unverifiable, and experiments show it adds cost without clear benefit.

Few-shot baseline (existing code in `CyberVisionAg/few_shot_eval.py`) is a separate comparison axis — no agent, just k random labeled images in context.

KB Coverage (Soybean, 32 classes total)

Source	Classes with data	Notable gaps
local	26/32	Diaporthe_2015_Kanawha, Green_stem_disorder, Stem_Canker, + 3
internet	23/32	Rhizoctonia, SCN, Downy_mildew, + 6

Coverage matters: classes without KB data are effectively in "none" mode regardless of source.

File Structure

```
CyberVisionAg/open_agentic/  
├── README.md           # This file – evolving plan + experiment log  
├── __init__.py         # Package marker  
├── eval.py             # Agentic harness: claude -p dispatch, metrics  
├── few_shot.py         # Few-shot baseline: single API call, no tools  
├── prompt.py           # Prompt construction (system + user message)  
└── run_eval.sh         # Quick eval launcher
```

Run commands

```
# From AgCrawler/ root:
source ~/miniconda3/etc/profile.d/conda.sh && conda activate vl-reasoning
set -a && source .env && set +a

# Exclude list for junk/duplicate classes
EXCLUDE="Diaporthe_2015_Kanawha,Green_stem,Fusarium_healthy_vs_infected,St
em_Canker,Top_Dieback"

# Quick smoke test (2 classes, 1 image each)
python -m CyberVisionAg.open_agentic.eval --symptom-source local --quick-
test 2

# 27-class eval (clean set) - agentic + local KB
PYTHONUNBUFFERED=1 python -m CyberVisionAg.open_agentic.eval \
  --symptom-source local --images-per-class 3 --k 4 --parallel 12 --seed
42 \
  --exclude "$EXCLUDE"

# 27-class eval - agentic + no KB
PYTHONUNBUFFERED=1 python -m CyberVisionAg.open_agentic.eval \
  --symptom-source none --images-per-class 3 --k 4 --parallel 12 --seed 42
\
  --exclude "$EXCLUDE"

# 27-class eval - few-shot baseline
PYTHONUNBUFFERED=1 python -m CyberVisionAg.open_agentic.few_shot \
  --images-per-class 3 --k 4 --parallel 12 --seed 42 \
  --exclude "$EXCLUDE"
```

Session recovery

This README is the single source of truth for continuing work if a conversation is lost. It contains:

- Current state of all experiments and findings
- Exact run commands (above)
- Phase checklist with completed/pending items
- Lessons learned that inform future experiments

To resume: read this README, check the latest experiment, and continue from the next item in the phase checklist.

Development Approach — Iterative with Feedback

This pipeline is built **one piece at a time**, tested at each step, with metrics driving the next change. The development cycle:

1. Make a small change (prompt tweak, architecture change)
2. Run minimal test (2 classes, 1 image each — ~30 seconds)
3. Check metrics (accuracy, turns, cost)
4. Record findings in this README under "Experiment Log"
5. Decide next change based on findings
6. Repeat

Rules for iteration:

- Never make multiple changes at once — isolate variables
- Always run the smoke test before committing a change
- Record ALL findings, including failures — they prevent repeating mistakes
- Update the plan section when direction changes
- Keep prompt changes versioned in this log

Implementation Phases

Phase 1: Minimal viable agent (done)

- ☒ `eval.py` — harness that spawns one `claude -p` call for a single test image
- ☒ `prompt.py` — construct the system/user prompt with test image, class list, KB, budget
- ☒ Parse `stream-json` output → extract prediction, trace, cost, turns
- ☒ Smoke test: 1 class, 1 image — produces valid prediction (Experiment 1)
- ☒ Add parallel dispatch (ThreadPoolExecutor, like current agent.py)
- ☒ Add CLI args matching current eval: `--symptom-source`, `--k`, `--num-classes`, `--images-per-class`, `--parallel`, `--seed`, `--quick-test`
- ☒ `run_eval.sh` — launcher script
- ☒ Smoke test: 3 classes, 1 image, parallel — works (Experiment 2-3)
- ☒ Filename leakage prevention (test images copied to neutral temp name)
- ☒ `--verbose` flag required with `stream-json` (discovered during Experiment 1)

Phase 2: KB comparison + failure analysis (done)

- ☒ 10-class eval with none/local/internet KB (Experiments 11-13)
- ☒ KB coverage tracking (local 8/10, internet 7/10)
- ☒ Failure analysis — visual ambiguity is the core challenge
- ☒ Budget experiments — more refs doesn't help (Experiments 8-10)
- ☒ Prompt experiments — prescriptive strategies don't help, reverted to minimal

Phase 3: Scale, baselines, and refinement (in progress)

- ☒ Few-shot baseline (Experiment 14) — agentic + local KB beats few-shot 47% vs 27%
- ☐ Dataset cleanup — remove duplicate/junk classes (see below), run on clean set
- ☐ Generate KB (local + internet xlsx) for clean dataset via disease_registry pipeline (see [disease_registry/README.md](#) for instructions)
- ☐ Full clean-dataset eval (none / local / internet + few-shot)
- ☐ Investigate Diaporthe problem — are test images genuinely ambiguous?
- ☐ Multi-reference per class (agent sees diverse training examples)

- ☐ Cost/accuracy tradeoff analysis

Dataset cleanup needed

The raw 32-class Soybean_Diseases has duplicates and junk:

Problem	Classes	Action
Duplicate disease	Diaporthe, Diaporthe_2015_Kanawha, Stem_Canker	Keep one (Diaporthe)
Duplicate name	Green_stem, Green_stem_disorder	Keep one (Green_stem_disorder)
Not a disease class	Fusarium_healthy_vs_infected	Remove (comparison, not disease)
Vague	Top_Dieback	Review — may overlap with other classes
Overlapping	Fusarium vs Fusarium_healthy_vs_infected	Keep Fusarium only

Clean set: ~26 classes. Copy to `Curated_Local_Dataset/{train,test}/Soybean_Clean/`, then generate KB xlsx files via the `disease_registry` pipeline. See [disease_registry/README.md](#) for pipeline instructions on building local (PDF) and internet (web) knowledge bases.

Experiment Log

Record every test run here. Format:

```
### Experiment N — [date] — [one-line description]
**Change**: what was changed
**Config**: classes/images/k/source
**Results**: accuracy, turns, cost
**Finding**: what was learned
**Next**: what to try next
```

Experiments 1-3 — 2026-03-14 — Pipeline bring-up

- Smoke tests (1 class, 3 classes, sequential + parallel)
- Validated: pipeline works, parallel works, predictions are reasonable
- Discovered: `--verbose` required with `stream-json`
- 5-class results: 72-80% accuracy (easy classes inflate this)

Experiments 4-7 — 2026-03-14 — 5-class eval + prompt tuning

- 5 classes too easy: Bacterial_Blight and Green_stem_disorder always 100%
- KB impact marginal at 5-class scale (all sources in 68-80% noise band)
- "Compare all candidates" prompt strategy: minor improvement, unreliable
- Lesson:** 5 classes inflates accuracy. Need harder test.

Experiments 8-10 — 2026-03-14 — 10-class scaling + budget tests

- Accuracy drops to ~37% with 10 classes (vs 72-80% with 5)
- Agent views only 2-3 refs regardless of budget (k=4 or k=10)
- Forced minimum refs (must view 5+): accuracy unchanged, just costs more
- **Lesson:** More reference views ≠ better accuracy. The bottleneck is reasoning quality, not data access.

Experiments 11-13 — 2026-03-14 — 10-class KB comparison (main result)

Config: 10 classes, 3 images each, k=4, parallel=12, seed=42

Source	Coverage	Accuracy	Cost/img	Avg turns	Avg refs
none	0/10	23% (7/30)	\$0.07	4.9	2.9
local	8/10	47% (14/30)	\$0.08	4.2	2.2
internet	7/10	37% (11/30)	\$0.07	4.1	2.1

Per-class breakdown:

Class	none	local	internet	local KB?	internet KB?
Anthracnose	0%	33%	33%	yes	yes
Bean_Pod_Mottle_virus	67%	33%	67%	yes	yes
Brown_Stem_Rot	33%	67%	33%	yes	yes
Diaporthe	0%	0%	0%	yes	yes
Diaporthe_2015_Kanawha	0%	0%	0%	NO	NO
Rhizoctonia	100%	100%	100%	yes	NO
Soybean_Cyst_Nematode	0%	33%	0%	yes	NO
Stem_Canker	0%	33%	0%	NO	yes
Tobacco_Streak_Virus	0%	67%	33%	yes	yes
White_Mold	33%	100%	100%	yes	yes

Key findings:

1. **KB nearly doubles accuracy** (23% → 47% with local)
2. **Local > Internet** (47% vs 37%) — PDF-quality descriptions + better coverage
3. **4 classes always 0%** (Diaporthe, D_2015_K) — visually ambiguous, resist all approaches
4. **2 classes always 100%** (Rhizoctonia, White_Mold) — visually distinctive regardless of KB
5. **KB helps most for medium-difficulty classes** (TSV 0→67%, White_Mold 33→100%, SCN 0→33%)
6. **Agent uses 2-3 refs and 4-5 turns** regardless of budget — self-regulates
7. **Cost nearly identical** across sources (~\$0.07-0.08/image)

Failure analysis (10-class)

Hard confusion patterns (visual ambiguity between diseases):

- **SCN, BSR, Diaporthe** → all show interveinal chlorosis. Agent defaults to BSR.
- **Diaporthe ↔ Phomopsis/Stem_Canker** → same genus, overlapping stem/pod symptoms.
- **TSV** → diverse presentations each resemble a different disease.

One ref image per class is limiting: each class shows ONE visual presentation. Diseases with variable symptoms (stems vs pods vs leaves) can't be captured by a single example.

Experiment 14 — 2026-03-14 — Few-shot baseline (10 classes)

Config: 10 classes, 3 images each, k=4 random labeled examples, parallel=12, seed=42

Approach	Accuracy	Cost/img
Few-shot (k=4 random)	27% (8/30)	\$0.02
Agentic + none (images only)	23% (7/30)	\$0.07
Agentic + internet KB	37% (11/30)	\$0.07
Agentic + local KB	47% (14/30)	\$0.08

Per-class comparison:

Class	Few-shot	Agentic none	Agentic local	Agentic internet
Anthracnose	0%	0%	33%	33%
Bean_Pod_Mottle_virus	67%	67%	33%	67%
Brown_Stem_Rot	67%	33%	67%	33%
Diaporthe	0%	0%	0%	0%
Diaporthe_2015_Kanawha	0%	0%	0%	0%
Rhizoctonia	67%	100%	100%	100%
Soybean_Cyst_Nematode	33%	0%	33%	0%
Stem_Canker	0%	0%	33%	0%
Tobacco_Streak_Virus	0%	0%	67%	33%
White_Mold	33%	33%	100%	100%

Key findings:

1. **Agentic + local KB beats few-shot by 20 percentage points** (47% vs 27%)
2. KB helps most for "medium" classes: TSV (0→67%), White_Mold (33→100%), BSR (33→67%)
3. Both Diaporthe classes remain at 0% across ALL approaches — genuinely hard
4. Few-shot is 4x cheaper per image (\$0.02 vs \$0.08) but significantly worse
5. Agentic without KB ≈ few-shot (23% vs 27%) — KB is the differentiator

Experiment 15-16 — 2026-03-14 — 27-class scale-up

Excluded 5 junk/duplicate classes (Diaporthe_2015_Kanawha, Green_stem, Fusarium_healthy_vs_infected, Stem_Canker, Top_Dieback) → 27 clean classes × 3 images = 81 test images.

Approach	Accuracy	Refs	Cost/img
Few-shot (k=4)	35% (28/81)	—	\$0.019
Agentic + local (no strategy)	37% (30/81)	1.4	\$0.087
Agentic + local (min refs)	38% (31/81)	3.3	\$0.089

Finding: At 27 classes, the agentic advantage over few-shot nearly vanished (37% vs 35%). Agent viewed only 1.4 refs — barely using its tools.

Investigation — 2026-03-14 — 6-agent parallel deep-dive

Spawned 6 agents to investigate 0% classes. Key findings:

- 1. **Overconfident misclassification:** Agent predicts wrong classes at 0.62-0.91 confidence. Visual observations are reasonable but mapped to wrong diseases.
- 2. **k/class ratio:** k=4 out of 27 classes = 15% coverage. Agent can't view enough references to discriminate.
- 3. **KB quality mismatch:** Descriptions emphasize textbook pathology, not visually distinctive features (e.g., White_Mold KB describes seeds, but diagnostic feature is stem mycelium).
- 4. **Only 5 training images per class:** Single "middle" reference ([_train_003.jpg](#)) may not be representative.
- 5. **Trace bug fixed:** Text was truncated to 500 chars, hiding agent reasoning.

Experiment 17 — 2026-03-14 — KB-guided strategy prompt

Change: System prompt now tells agent to: (1) read test image, (2) use KB to narrow candidates, (3) view references only for top candidates, (4) submit. This makes the agent use KB strategically before spending its reference budget.

Approach	Accuracy	Refs	Cost/img
Few-shot baseline	35% (28/81)	—	\$0.019
Agentic + local (no strategy)	37% (30/81)	1.4	\$0.087
Agentic + local (KB-guided)	48% (39/81)	3.1	\$0.109

+10pp improvement from strategy prompt alone. Classes that improved: BSR 0→67%, SCN 0→67%, Rhizoctonia 0→67%, Bacterial_Pustule 0→33%, BPMV 0→33%, White_Mold 0→33%.

Still at 0% (5 classes): Cercospora, Fusarium, Phomopsis, Phyllosticta_leaf_spot, Soybean_Dwarf_Mosaic_Virus, Soybean_Vein_necrosis_virus, Tobacco_Streak_Virus.

Current best result: 48% on 27 classes (agentic + local KB + strategy prompt)

This is **+13pp over few-shot** (48% vs 35%) on the same model, same budget.

Experiment 18 — 2026-03-14 — Compact ref paths (reverted)

Change: Replaced 27 full paths with a pattern template to reduce prompt noise **Results:** 38% (31/81) — dropped from 48%. But logs showed refs=3-4 with no errors, so paths weren't the issue. Likely LLM stochasticity. Reverted to full paths. **Finding:** Don't change what works without evidence. The 48% vs 38% difference (8 images on 81 total) is within noise range.

Experiment 19 — 2026-03-14 — Parallel scaling

Change: Increased parallel from 12 to 20, then 16 **Results:** parallel=20 → 23/81 errors (rate limiting). parallel=16 → 81/81 errors (credit exhaustion). **Finding:** parallel=12 is the safe limit. Higher causes API errors. Credits depleted after ~\$50 total spend.

Current status (2026-03-14)

Best result: **48% accuracy** on 27 classes with agentic + local KB + strategy prompt, vs **35% few-shot baseline** (+13pp). Typical range is 37-48% due to LLM stochasticity.

Fast iteration principle

The feedback loop must be fast. Every experiment should complete in ~2 minutes so we can iterate quickly. The standard fast config is:

- **27 classes × 2 images/class = 54 test images** (instead of 3 images × 27 = 81)
- Covers ALL classes (catches regressions everywhere) while being ~1.5x faster and cheaper
- At parallel=12, runs in ~2 minutes and costs ~\$5
- Each flip is ~1.9pp noise — stable enough to detect real signal
- Use 3 images/class only for final validation of a promising change

```
# Fast feedback run (27 classes × 2 images = 54 tests, ~2 min)
PYTHONUNBUFFERED=1 python -m CyberVisionAg.open_agentic.eval \
  --symptom-source local --images-per-class 2 --k 4 --parallel 12 --seed
42 \
  --exclude "$EXCLUDE"

# Full validation run (only after fast run shows improvement)
PYTHONUNBUFFERED=1 python -m CyberVisionAg.open_agentic.eval \
  --symptom-source local --images-per-class 3 --k 4 --parallel 12 --seed
42 \
  --exclude "$EXCLUDE"
```

Experiment 20 — 2026-03-14 — Fast baseline (27×2)

Change: Established fast feedback config — 27 classes × 2 images = 54 tests **Config:** local KB + strategy prompt, k=4, parallel=12, seed=42, exclude 5 junk classes **Results:** **42.6%** (23/54), 0 errors, avg 5.1 turns, 3.1 refs, \$0.10/img, \$5.57 total, ~4.5 min wall time

Per-class:

100% (7 classes)	50% (7 classes)	0% (13 classes)
Brown_Stem_Rot, Diaporthe, Frogeye_leaf_spot, Green_stem_disorder, Powdery_Mildew, Purple_Seed_Stain, Pythium_damping_off	Bacterial_Blight, Phyllosticta_leaf_spot, Phytophthora, Rhizoctonia, SCN, SDS, White_Mold	Anthracnose, Bacterial_Pustule, BPMV, Cercospora, Charcoal_Rot, Downy_mildew, Fusarium, Phomopsis, Septoria, SDMV, SVN, Soybean_rust, TSV

Finding: 42.6% confirms the 37-48% range from prior experiments. 0 errors (credits working). Fast config takes ~4.5 min — acceptable for iteration. 13 classes at 0% with only 2 images means some are just unlucky (e.g., Anthracnose was 33-100% in prior runs), but the persistent 0% classes (Fusarium, SDMV, SVN, TSV) are real failures.

KB coverage observation: Currently only Visual Description (col E) is sent to the agent. The xlsx also has Pathogen, Type of Disease, Affected Parts columns — internet KB fills all of these, local KB only fills Affected Parts. Sending more columns (especially Affected Parts and disease Type) could help the agent narrow candidates faster.

Experiment 21 — 2026-03-14 — Internet KB: all columns vs Visual Description only (A/B)

Change: Modified `_load_xlsx_as_markdown()` to optionally include Pathogen, Type, Affected Parts (via `--all-kb-columns` flag). Ran internet KB both ways for a clean comparison.

Config: internet KB, 27x2, k=4, parallel=12, seed=42

Variant	Accuracy	Cost/img
Visual Description only (control)	44.4% (24/54)	\$0.098
All columns (Pathogen+Type+Parts+Desc)	42.6% (23/54)	\$0.106

Per-class differences (only showing changes):

Class	Desc only	All cols	Delta
Bacterial_Blight	100%	50%	-50
Bacterial_Pustule	0%	50%	+50
Cercospora	0%	50%	+50
Charcoal_Rot	0%	50%	+50
Diaporthe	0%	0%	—
Fusarium	50%	0%	-50
SCN	50%	0%	-50

Finding: Adding Pathogen/Type/Affected Parts metadata **did not help** — 1 image difference (within noise). The extra context slightly increases cost (+8%) without accuracy gain. Visual Description alone carries the useful signal. Dropping this line of investigation.

Cross-source comparison (all at 27x2, seed=42):

KB Source	Accuracy	Cost/img
Local (Exp 20)	42.6% (23/54)	\$0.103
Internet — desc only	44.4% (24/54)	\$0.098
Internet — all cols	42.6% (23/54)	\$0.106

All three are within the ~42-44% noise band. KB source and metadata richness are not the bottleneck.

Experiment 22 — 2026-03-15 — KB improvement loop (clean)

Setup: Built `improve_kb.py` — a `claude -p` agent that analyzes confusion patterns + reasoning traces and rewrites KB descriptions with differential features. Added `--kb-file` flag to `eval.py`.

Bug fixed: `eval.py` now clears the log directory before each run, so the improver only sees results from the current eval (previously stacked results from multiple runs).

Approach: "Only-failing" mode — improver rewrites descriptions only for classes < 100%, copies 100% classes verbatim to avoid whack-a-mole.

Loop: v0 (internet baseline) → improve → v1_clean → eval

KB Version	Accuracy	Cost/img	Notes
v0 (internet baseline, clean)	42.6% (23/54)	\$0.098	6/27 classes have no KB entry
v1_clean (only-failing improved)	42.6% (23/54)	\$0.101	Improver also generated entries for 5 missing diseases

Per-class: 100% classes all preserved (whack-a-mole fixed). But still 2 regressions and 2 improvements at 50% level, netting zero.

Key findings from deep investigation:

- Differential KB WORKS where it applies** — reasoning traces confirm the agent uses "Unlike X..." distinctions correctly (e.g., Charcoal_Rot vs Phytophthora: "gray with specks" vs "chocolate-brown" distinction)
- KB coverage gap:** Internet source has NO data for 6/27 classes (Downy_mildew, Rhizoctonia, SCN, SDMV, SVN, Soybean_rust). These classes run in effectively "no KB" mode.
- 2 images/class is too noisy for improvement signal** — each flip is ±50pp per class, making it impossible to distinguish real improvement from stochasticity
- Accuracy is stable at ~42-44%** across 4 runs with different KB variants. This appears to be the capability ceiling for the current architecture (1 ref image per class, sonnet model).
- The improver correctly generates new entries** for missing diseases, but with 2 images these can't be validated

File structure update

```
CyberVisionAg/open_agentic/
├── README.md           # This file
├── __init__.py
├── eval.py             # Agentic harness with --kb-file support
├── few_shot.py         # Few-shot baseline
├── improve_kb.py       # KB improver: analyzes failures, rewrites
descriptions
├── prompt.py          # Prompt construction
├── run_eval.sh         # Quick eval launcher
├── kb_v0.md            # Original internet KB (Visual Description only)
├── kb_v1.md            # First improvement (stale data – superseded)
└── kb_v1_clean.md     # Clean improvement (only-failing, 54 results)
```

Experiment 23 — 2026-03-15 — Multi-reference images (3/class)

Change: Added `--refs-per-class` flag. Agent now sees 3 evenly-spaced training images per class (was 1 middle image). Increased k from 4 to 6 to give budget for viewing more refs.

Config: internet KB, 27×2, refs-per-class=3, k=6, parallel=12, seed=42

Refs/class	Accuracy	Avg refs viewed	Cost/img
1 (baseline)	42.6% (23/54)	3.2	\$0.098
3	40.7% (22/54)	5.3	\$0.141

Finding: More reference images did not help — 40.7% vs 42.6% (within noise). Agent viewed 5.3 refs (vs 3.2), so it did use the extra refs, but accuracy didn't benefit. Cost increased 43%. The bottleneck is not reference image quantity.

Summary of all experiments at 27×2

Experiment	Change	Accuracy	Cost/img
20	Local KB (baseline)	42.6%	\$0.103
21	Internet KB (desc only)	44.4%	\$0.098
21	Internet KB (all cols)	42.6%	\$0.106
22	Internet KB, clean baseline	42.6%	\$0.098
22	Improved KB v1 (differential)	42.6%	\$0.101
23	Multi-ref (3/class, k=6)	40.7%	\$0.141

All within 40-44%. This is the capability ceiling for the current architecture.

Experiment 24 — 2026-03-15 — Model comparison (Haiku vs Sonnet)

Change: Added `--model` CLI flag. Ran Haiku to test if model quality matters.

Config: internet KB, 27×2, k=4, parallel=12, seed=42, refs-per-class=1

Model	Accuracy	Cost/img	Avg duration
Haiku	25.9% (14/54)	\$0.049	29s
Sonnet	42.6% (23/54)	\$0.098	46s

Finding: Haiku drops 17pp — model capability IS a significant factor. The 42% ceiling is not inherent to the task or KB, it's partly a model limitation. Opus is worth testing.

Experiment 25 — 2026-03-15 — Opus

Config: internet KB, 27x2, k=4, parallel=12, seed=42, refs-per-class=1

Model	Accuracy	Cost/img	Duration	100% classes
Haiku	25.9% (14/54)	\$0.049	29s	2
Sonnet	42.6% (23/54)	\$0.098	46s	6
Opus	46.3% (25/54)	\$0.166	40s	8

Opus 100% classes: Anthracnose, Bacterial_Pustule, Frogeye, Green_stem_disorder, Phomopsis, Phytophthora, Purple_Seed_Stain, Pythium_damping_off, White_Mold (9 actually — plus Brown_Stem_Rot at 50%).

Finding: Clear model quality scaling — Haiku 26% → Sonnet 43% → Opus 46%. Diminishing returns from Sonnet→Opus (+3.7pp, 70% more expensive) vs Haiku→Sonnet (+17pp, 2x cost). Opus helps on a few specific classes (Phomopsis 0→100%, Bacterial_Pustule 0→100%) but the hard 0% classes (BPMV, Cercospora, Diaporthe, Fusarium, Septoria, SDMV, SVN, Soybean_rust, SDS, TSV) remain at 0% across all models.

Planned: Model x K sweep

Goal: Full comparison matrix — Model (haiku, sonnet, opus) x K (1, 2, 4, 8), internet KB fixed.

Script: `run_model_k_sweep.sh` — runs 12 experiments, writes `sweep_results.csv`.

```
cd /path/to/AgCrawler
source ~/miniconda3/etc/profile.d/conda.sh && conda activate vl-reasoning
set -a && source .env && set +a
bash CyberVisionAg/open_agentic/run_model_k_sweep.sh
```

Status: Credits exhausted (2026-03-15). Script ready — run when credits available.

Future sweeps (same script, change SOURCE variable):

- ☐ Internet KB sweep (current)
- ☐ Local KB sweep
- ☐ No KB sweep

Next steps

- ☐ Run `run_model_k_sweep.sh` when credits available
- ☐ Repeat sweep with local KB and none KB
- ☐ KB improvement loop — parked for now, revisit after sweep results

Claude -p Reference

See [claude-headless.md](#) for `claude -p` invocation patterns, output parsing, and environment setup.

Key gotchas from existing codebase (`shared.py`):

- Must strip `CLAUDE*`, `CURSOR*`, `MCP_CONNECTION*`, `VSCODE*`, `ELECTRON*` env vars — otherwise `claude -p` hangs (nested session detection)
- Use `Popen` with `start_new_session=True` + `os.killpg()` for reliable timeout/cleanup
- `--output-format stream-json` gives NDJSON with `assistant` events (tool calls, text) and a final `result` event with `num_turns`, `total_cost_usd`, `duration_ms`